JAVA 2

Module FX - première partie

Une fenêtre qui réagit

Au programme

1. Le lancement
2. La fenêtre
3. Les gestionnaires
4. Les événements
5. La propagation

1. Lancer une application JavaFX



JavaFX n'est plus dans le JDK

Depuis Java 11, JavaFX est une bibliothèque à part

Le projet du cours la déclare dans son pom.xml, comme JUnit

Maven la télécharge : rien à installer, rien à configurer

```
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21.0.2</version>
</dependency>
```

La classe à lancer s'appelle `Lanceur`

Une application `JavaFX` hérite de `Application`

Lancée directement, elle refuse de démarrer

On lance une autre classe, qui n'hérite de rien et appelle `launch`

```
public class LanceurLampe {

    public static void main(String[] args) {
        Application.launch(Lampe.class, args);
    }
}
```

Si rien ne démarre

« `JavaFX runtime components are missing` » : rien ne manque

Vous avez lancé la classe d'application au lieu du `Lanceur`

En secours : `javafx:run`, dans la fenêtre Maven (guide IntelliJ, `IDE.28`)

2. Une fenêtre et ses composants



Qui appelle `start` ?

La méthode `launch` crée l'objet `Application`, puis appelle `start`

`JavaFX` fournit la fenêtre, et la passe en paramètre

La méthode `start` construit l'écran, l'affiche, et se termine

La fenêtre, elle, reste ouverte

```
public class Lampe extends Application {

    @Override
    public void start(Stage fenetre) {
        Label etat = new Label("Eteinte");
        fenetre.setScene(new Scene(new StackPane(etat), 300, 150));
        fenetre.setTitle("Lampe");
        fenetre.show();
    }
}
```

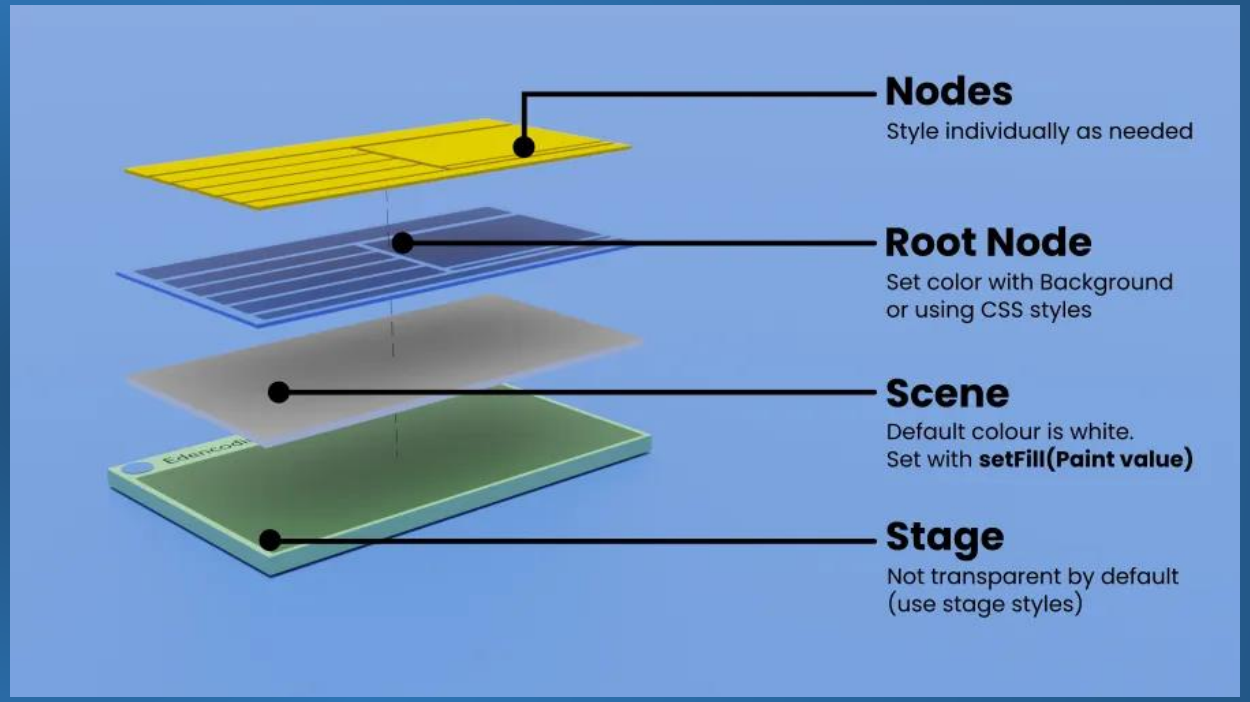
Le graphe de scène

Une fenêtre (`Stage`) affiche une scène (`Scene`)

La scène contient une seule racine

Un conteneur range des enfants, qui peuvent être des conteneurs

L'ensemble forme un arbre de nœuds (`Node`)



Construire le graphe de scène

On crée les nœuds, puis le conteneur qui les range, puis la scène autour de la racine

`getChildren()` retourne la liste des enfants : on peut y ajouter après coup

```
Label etat = new Label("Eteinte");
Button interrupteur = new Button("Interrupteur");

VBox racine = new VBox(etat, interrupteur);
Scene scene = new Scene(racine, 300, 150);
```

Création des nœuds

Création du conteneur

Création de la Scene

```
// Un enfant peut s'ajouter apres coup
racine.getChildren().add(new Label("Bureau"));
```

Ajout d'un enfant après coup

Les conteneurs

Conteneur	Range ses enfants	Usage typique
`HBox`	côte à côte	une rangée de boutons
`VBox`	les uns sous les autres	un formulaire
`BorderPane`	en cinq zones	une application
`StackPane`	empilés et centrés	un texte sur une forme

Les contrôles

Un `Label` affiche un texte que l'utilisateur ne modifie pas

Un `TextField` reçoit une saisie, que `getText` relit

Un `Button` déclenche une action quand on l'actionne

```
Label consigne = new Label("Piece :");
TextField saisie = new TextField("Bureau");
Button valider = new Button("Valider");

saisie.setPromptText("ex. Cuisine");
String piece = saisie.getText();
saisie.setEditable(false);
```

3. Brancher un bouton



Le programme attend

Un programme console décide de l'ordre dans lequel les choses arrivent

Une interface graphique attend que l'utilisateur agisse

Chaque geste produit un événement

JavaFX le transmet au code qui s'est abonné à l'événement, et à lui seul

Démonstration



Démonstration

Le bouton qui ne fait rien

Déroulé : Demo_FX

Un gestionnaire est un objet

L'interface `EventHandler` déclare une seule méthode, `handle`

Elle est fonctionnelle : une lambda suffit à en créer un objet

Le bouton garde cet objet, et appelle `handle` à chaque clic

```
// javafx.event.EventHandler, simplifiee
public interface EventHandler<T extends Event> {
    void handle(T evenement);
}
```

```
// Le bouton recoit un objet qui implemente l'interface
interrupteur.setOnAction(evenement -> basculer());
```

setOnAction
`public final void setOnAction(EventHandler<ActionEvent> value)`

`setOnAction` n'est qu'un raccourci

La méthode `addEventHandler` nomme le type d'événement écouté

Les gestionnaires ajoutés s'additionnent

Pour en retirer un, `removeEventHandler` veut le même objet : on le range dans un champ

```
private final EventHandler<ActionEvent> basculement =
        evenement -> basculer();
```

```
private void verrouiller() {
    interrupteur.removeEventHandler(ActionEvent.ACTION, basculement);
}

private void deverrouiller() {
    interrupteur.addEventHandler(ActionEvent.ACTION, basculement);
}
```

Exercices



FX.1 Un compteur de participants

fascicule FX

FX.2 Un bouton Reset

fascicule FX

FX.3 Deux compteurs, et une mise en page

fascicule FX

4. Ce que dit un événement



Démonstration



Démonstration

Une touche frappée

Déroulé : Demo_FX

Ce que transporte un événement

Le gestionnaire reçoit l'événement en paramètre

La méthode `getSource` retourne le composant dont le gestionnaire s'exécute

La méthode `getEventType` dit ce qui s'est passé

Un même gestionnaire peut alors servir à plusieurs composants

```
// dans start
allumer.setOnAction(this::noterBouton);
eteindre.setOnAction(this::noterBouton);
```

```
private void noterBouton(ActionEvent evenement) {
    Button presse = (Button) evenement.getSource();
    journal.setText("Dernier bouton : " + presse.getText());
}
```

Les familles d'événements

Classe	Produit par	Types utiles ici
`ActionEvent`	un bouton actionné	`ACTION`
`KeyEvent`	le clavier	`KEY_PRESSED`, `KEY_TYPED`
`MouseEvent`	la souris	`MOUSE_CLICKED`
`WindowEvent`	la fenêtre	`WINDOW_CLOSE_REQUEST`

Exercices



FX.4 Compter au clavier

fascicule FX

À vous de chercher



La touche « flèche haut » ne produit aucun caractère

Comment votre programme peut-il savoir qu'on l'a enfoncée ?

Cherchez dans la Javadoc de `KeyEvent` : comparez `KEY_TYPED` et `KEY_PRESSED`

Une touche, deux événements



Le type `KEY_TYPED` donne un caractère, par `getCharacter`

Le type `KEY_PRESSED` donne une touche, par `getCode`, un `KeyCode`

Une flèche n'a pas de caractère : seul `KEY_PRESSED` la voit

```
racine.addEventHandler(KeyEvent.KEY_PRESSED, this::toucheEnfoncee);
```

```
private void toucheEnfoncee(KeyEvent evenement) {
    switch (evenement.getCode()) {
        case UP -> augmenterIntensite();
        case DOWN -> diminuerIntensite();
        default -> { }
    }
}
```

La souris



Le type `MOUSE_CLICKED` survient quand on presse puis relâche sur le même nœud

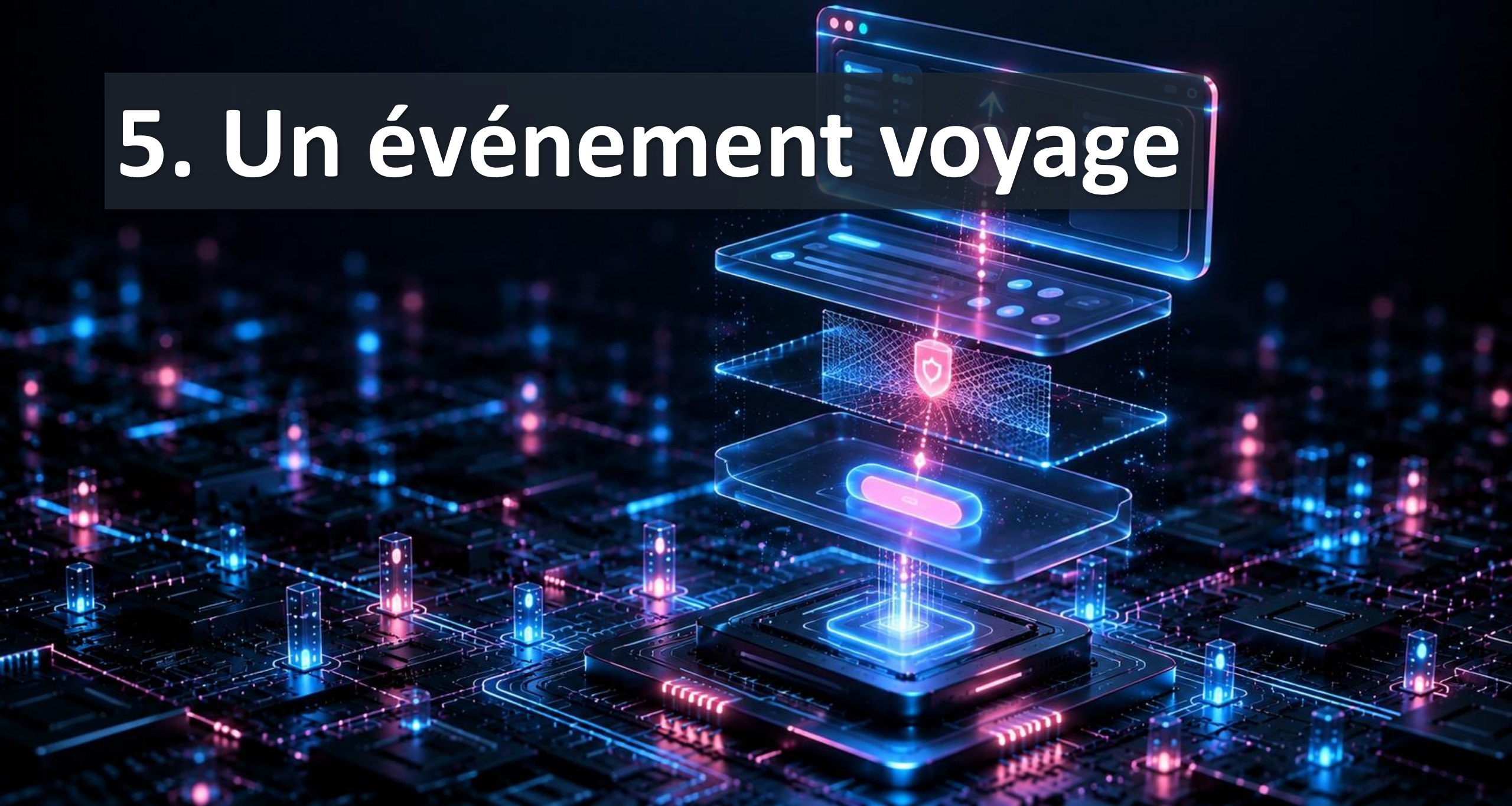
Les méthodes `getX` et `getY` donnent la position dans le nœud

La méthode `getClickCount` distingue le simple clic du double clic

La méthode `getButton` dit quel bouton de la souris a servi

```
ampoule.addEventHandler(MouseEvent.MOUSE_CLICKED, evenement -> {
    if (evenement.getClickCount() == 2) {
        basculer();
    }
});
```

5. Un événement voyage



Démonstration



Démonstration

Un événement voyage

Déroulé : Demo_FX

L'aller et le retour

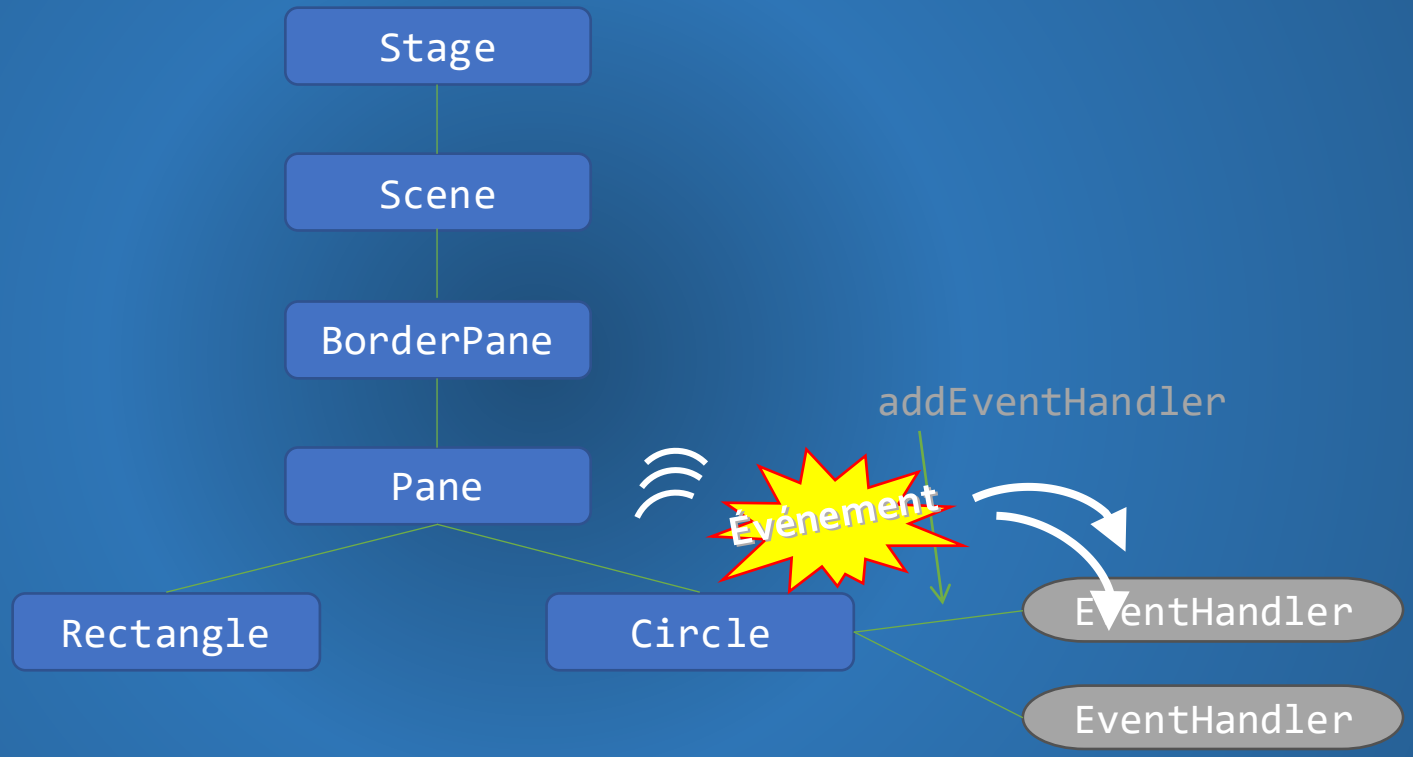
JavaFX trace d'abord le chemin, de la fenêtre jusqu'au nœud visé

L'événement descend ce chemin : c'est la capture

Il atteint la cible, puis remonte le même chemin : c'est le bouillonnement

Chaque nœud traversé peut intervenir, à l'aller comme au retour

La propagation des événements

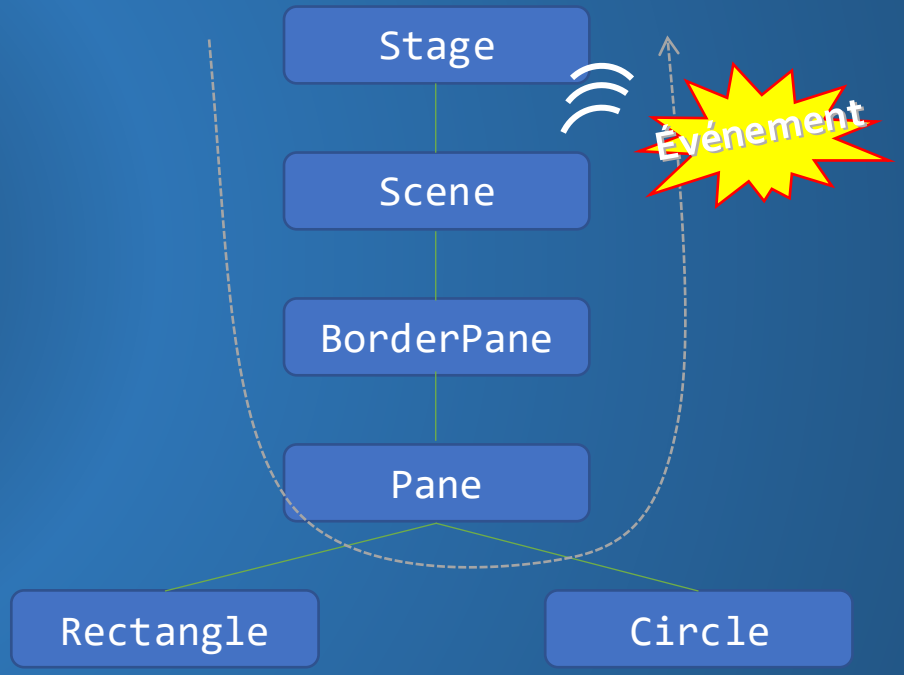


Source : Stack Overflow, question 25740103

Les `EventFilter`

Lors du déclenchement d'un événement, le processus qui détermine quel Node traite l'événement se déroule en deux phases :

- event capturing (descente)
- event bubbling (remontée)

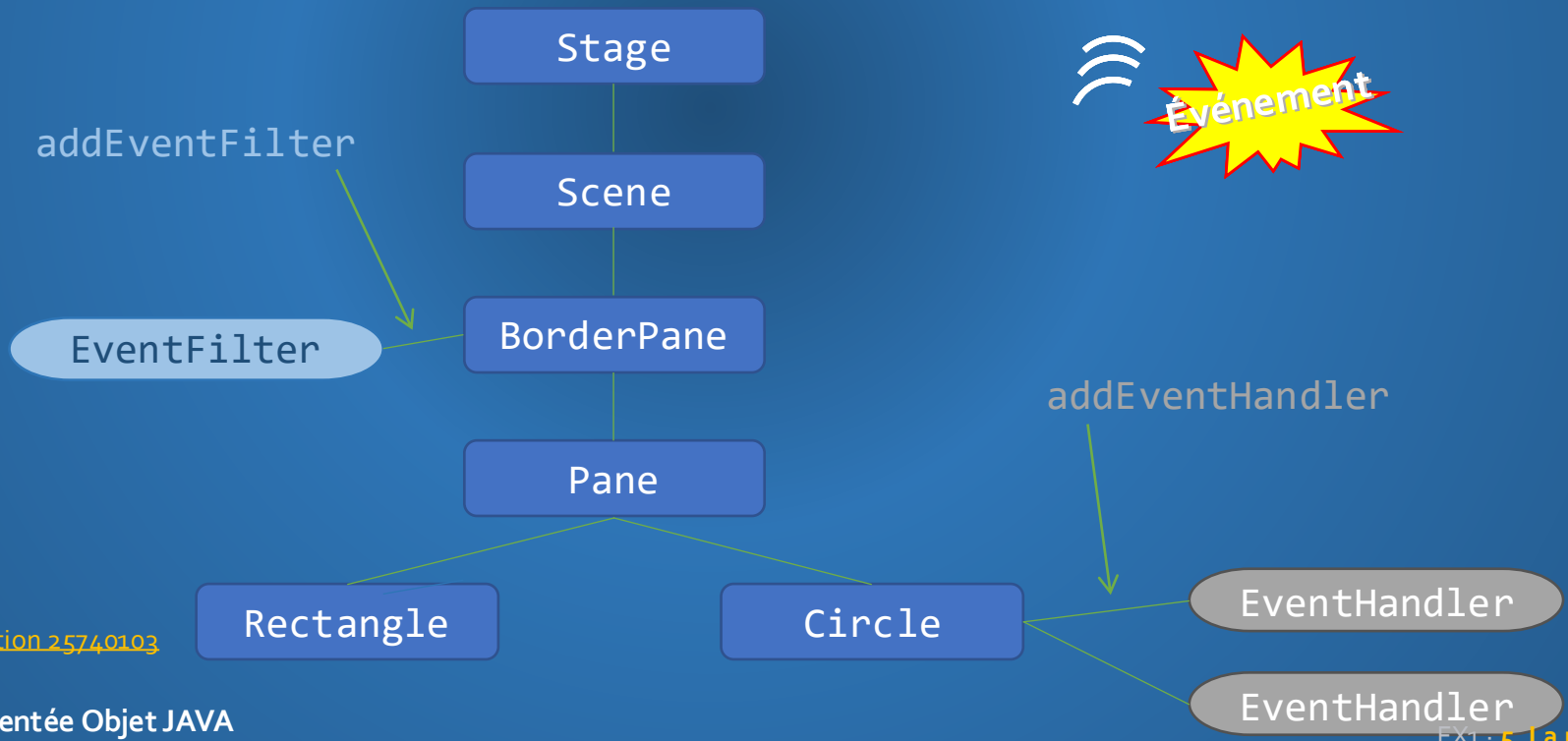


Source : Stack Overflow, question 25740103

EventHandler et EventFilter

Deux types d'objets traitent les `Event`:

- Les `EventFilters` lors de la descente.
- Les `EventHandlers` lors de la remontée.



Source : Stack Overflow, question 25740103

Filtre ou gestionnaire

	Filtre	Gestionnaire
Phase	la descente	la remontée
Ordre	du haut vers la cible	de la cible vers le haut
Ajout	`addEventFilter`	`addEventHandler`
Sert à	intercepter avant les enfants	réagir à ce qui s'est passé

consume()

La méthode `consume` marque l'événement comme traité

Les nœuds suivants du chemin ne le reçoivent plus

Consommé par un filtre, il n'atteint jamais la cible

Les autres gestionnaires du même nœud sont, eux, encore appelés

```
panneau.addEventFilter(MouseEvent.MOUSE_CLICKED, evenement -> {
    if (verrouille) {
        evenement.consume();
    }
});
```

Exercices



FX.5 Un événement voyage

fascicule FX

Le fil rouge



G.33 Le Nim dans une fenêtre

fil rouge GameHub

Les classes du Nim servent telles quelles, sauf sa boucle